

Exercise 1a: Forward Kinematics of the ABB IRB 120

Prof. Marco Hutter*

Teaching Assistants:

Arjun Bhardwaj, Thomas Li, Ennio Schnieder

September 23, 2026



Figure 1: The ABB IRW 120 robot arm.

Abstract

The aim of this exercise is to calculate the forward and inverse kinematics of an ABB robot arm. In doing so, you will practice the use of different representations of the end-effector's orientation as well as how to check whether your implementations are correct. Essentially, the task is to implement the functions for computing the forward and inverse kinematics using symbolic and numerical computations in MATLAB. A separate MATLAB script will be provided for the 3D visualization of the robot arm.

*original contributors include Michael Blösch, Dario Bellicoso, and Samuel Bachmann

1 Introduction

The following exercise is based on an ABB IRB 120 depicted in Figure 2. It is a 6-link robotic manipulator with a fixed base. During the exercise you will implement several different MATLAB functions, which you should test carefully since the following tasks are often dependent on them. To help you with this, use the provided script prototypes (download from Moodle).

2 Forward Kinematics

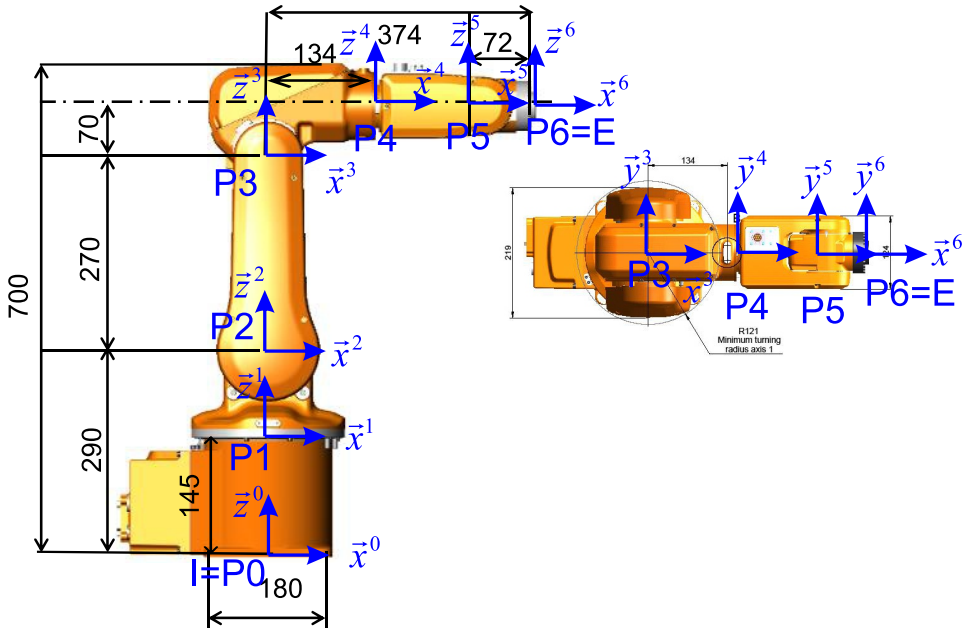


Figure 2: ABB IRB 120 with coordinate systems and joints. The units are mm.

Throughout this document, we will employ I for denoting the inertial world coordinate system (coordinate system P0 in Figure 2) and E for the coordinate system attached to the end-effector (coordinate system P6 in Figure 2).

You should always check your solutions with the provided script `evaluate_problems.m`. This script compares your implementation with our solution on random data points.

Exercise 2.1

Define a vector $\mathbf{q}$ of generalized coordinates to describe the configuration of the ABB IRB120. Recall that generalized coordinates should be *complete* and *independent*. The former property means that they should fully describe the configuration of the robot while at the same time comprising a minimal set of coordinates. The latter property refers to the fact that the each generalized coordinate must not be a function of any of the others.

Exercise 2.2

Assume from here on that the generalized coordinates $\mathbf{q}$ are the joint angles of the robot arm numbered according to Figure 2. Positive angles imply rotations around the positive coordinate axis.

Compute the homogeneous transformations matrices $\mathbf{T}_{k-1,k}(q_k)$, $\forall k = 1, \dots, 6$. Additionally, find the constant homogeneous transformations between the inertial

frame and frame 0 ($\mathbf{T}_{I0}$) and between frame 6 and the end-effector frame ($\mathbf{T}_{6E}$). Please implement the following functions (i.e., replace the zero assignments with your solution):

```

1 function TI0 = getTransformI0()
2 % Input: void
3 % Output: homogeneous transformation Matrix from frame 0 to the ...
4 %           inertial frame I. T_I0
5 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
6 TI0 = zeros(4);
7 end
8
9 function T01 = jointToTransform01(q)
10 % Input: joint angles
11 % Output: homogeneous transformation Matrix from frame 1 to frame ...
12 %           0. T_01
13 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
14 T01 = zeros(4);
15 end
16
17 function T12 = jointToTransform12(q)
18 % Input: joint angles
19 % Output: homogeneous transformation Matrix from frame 2 to frame ...
20 %           1. T_12
21 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
22 T12 = zeros(4);
23 end
24
25 function T23 = jointToTransform23(q)
26 % Input: joint angles
27 % Output: homogeneous transformation Matrix from frame 3 to frame ...
28 %           2. T_23
29 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
30 T23 = zeros(4);
31 end
32
33 function T34 = jointToTransform34(q)
34 % Input: joint angles
35 % Output: homogeneous transformation Matrix from frame 4 to frame ...
36 %           3. T_34
37 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
38 T34 = zeros(4);
39 end
40
41
42 function T45 = jointToTransform45(q)
43 % Input: joint angles
44 % Output: homogeneous transformation Matrix from frame 5 to frame ...
45 %           4. T_45
46 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
47 T45 = zeros(4);
48 end
49
50 function T56 = jointToTransform56(q)
51 % Input: joint angles
52 % Output: homogeneous transformation Matrix from frame 6 to frame ...
53 %           5. T_56
54 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION

```

```

55     T56 = zeros(4);
56 end
57
58 function T6E = getTransform6E()
59     % Input: void
60     % Output: homogeneous transformation Matrix from the end-effector ...
        frame E to frame 6. T_6E
61
62     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
63     T6E = zeros(4);
64 end

```

Exercise 2.3

Find the end-effector position vector ${}^I\mathbf{r}_{IE} = {}^I\mathbf{r}_{IE}(\mathbf{q})$. Please implement the following function:

```

1 function I_r_IE = jointToPosition(q)
2     % Input: joint angles
3     % Output: position of end-effector w.r.t. inertial frame. I_r_IE
4
5     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
6     I_r_IE = zeros(3,1);
7 end

```

Exercise 2.4

What is the end-effector position for $\mathbf{q} = \begin{pmatrix} \pi/6 \\ \pi/6 \\ \pi/6 \\ \pi/6 \\ \pi/6 \\ \pi/6 \end{pmatrix}$?

Use Matlab (`abbRobot.setJointPositions(q)`) to visualize it.

Exercise 2.5

Find the end-effector rotation matrix $\mathbf{C}_{IE} = \mathbf{C}_{IE}(\mathbf{q})$. Please implement the following function:

```

1 function C_IE = jointToRotMat(q)
2     % Input: joint angles
3     % Output: rotation matrix which projects a vector defined in the
        end-effector frame E to the inertial frame I, C_IE.
4
5
6     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
7     C_IE = zeros(3);
8 end

```

Exercise 2.6

Find the quaternion representing the attitude of the end-effector $\xi_{IE} = \xi_{IE}(\mathbf{q})$. Please also implement the following function:

- Two functions for converting from quaternion to rotation matrices and vice-versa. Test these by converting from quaternions to rotation matrices and back to quaternions.
- The quaternion multiplication $\mathbf{q} \otimes \mathbf{p}$
- The passive rotation of a vector with a given quaternion. This can be implemented in different ways which can be tested with respect to each other.

The easiest way is to transform the quaternion to the corresponding rotation matrix (by using the function from above) and then multiply the matrix with the vector to be rotated.

Also check that your two representations for the end-effector orientation match with each other. In total you should write the following five functions:

```

1 function quat = jointToQuat(q)
2     % Input: joint angles
3     % Output: quaternion representing the orientation of the end-effector
4     % q_IE.
5
6     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
7     quat = zeros(4,1);
8 end
9
10 function R = quatToRotMat(q)
11     % Input: quaternion [w x y z]
12     % Output: corresponding rotation matrix
13
14     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
15     R = zeros(3);
16 end
17
18 function q = rotMatToQuat(R)
19     % Input: rotation matrix
20     % Output: corresponding quaternion [w x y z]
21
22     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
23     q = zeros(4,1);
24 end
25
26 function q_AC = quatMult(q_AB,q_BC)
27     % Input: two quaternions to be multiplied
28     % Output: output of the multiplication
29
30     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
31     q_AC = zeros(4,1);
32 end
33
34 function B_r = rotVecWithQuat(q_BA,A_r)
35     % Input: the orientation quaternion and the coordinate of the ...
36             vector to be mapped
37     % Output: the coordinates of the vector in the target frame
38
39     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
40     B_r = zeros(3,1);
41 end

```